DOCUMENTATION TECHNIQUE

Documentation technique — Concert Agent

Architecture, tools, base SQLite, sécurité, tests et choix techniques.

CA

Concert Agent · Octobre 2026

Cette documentation explique **comment l'application fonctionne**, même si tu n'as jamais programmé un agent LLM. Les termes techniques sont définis au début, puis chaque partie décrit un mécanisme avec un exemple concret.

---

1. Objectif et public

Concert Agent est une application **Flask** qui répond à une demande en français en choisissant et en exécutant des **tools Python**. Les concerts viennent de l'API **Ticketmaster** ou, à défaut, d'un jeu de démonstration étiqueté. L'agent ne fabrique jamais de date, de salle ou de prix.

Le projet remplace l'ancien site Node `concert-alert` : la connexion Spotify, les favoris, l'historique et les traces sont désormais dans le même backend Flask et la même base SQLite.

Comment lire ce document

Ton objectif	Lis en priorité
Comprendre le projet en 10 minutes	Sections 2 à 4
Vérifier que le projet respecte le devoir	Sections 5, 6, 13 et 16
Installer ou déployer	`README.md`, puis section 15
Comprendre une erreur	Section 17 (FAQ)

2. Vocabulaire

Mot	Définition simple
LLM	Modèle de langage qui comprend une demande et produit du texte ou une demande d'action. Ici : Gemini.
Agent	Programme qui enchaîne plusieurs étapes : comprendre, appeler un tool, lire le résultat, répondre.
Tool	Fonction Python que l'agent peut appeler. Exemple : `rechercher_concerts`.
Prompt système	Consignes générales envoyées au LLM avant la conversation.
Tool call	Demande structurée du LLM pour exécuter un tool avec des arguments.
SQLite	Base de données légère, stockée dans un fichier, qui garde les données entre deux requêtes.
Trace	Enregistrement d'un appel de tool : nom, arguments, résultat, statut, durée.
Action en attente	Écriture préparée mais pas encore exécutée, stockée dans `pending_actions`.
Fallback	Solution de secours : mode règles sans LLM, ou données de démonstration sans Ticketmaster.
OAuth	Procédure officielle de connexion à Spotify sans stocker le mot de passe de l'utilisateur.
API	Service externe interrogé par HTTP. Ici : Ticketmaster et Spotify.
Cache	Stockage temporaire d'un résultat pour éviter de le recalculer immédiatement.

3. Vue d'ensemble



Cinq couches, avec une responsabilité par couche :

1. **Interface** : `templates/index.html`, `static/app.js`, `static/style.css`.
2. **Serveur** : `app.py` reçoit les requêtes HTTP et gère la session Flask.
3. **Orchestration** : `agent/loop.py` fait tourner la boucle de l'agent.
4. **Tools** : `agent/tools.py` parle à Ticketmaster et utilise la base.
5. **Mémoire** : `db.py` et SQLite gardent les préférences, favoris et traces.

Le LLM n'a **pas** accès directement à SQLite, à Ticketmaster ou au système de fichiers. Il ne peut agir qu'en demandant un tool autorisé.

---

4. Cycle d'une demande

Version racontée

L'utilisateur écrit :

> « Trouve un concert de Gims à Décines. »

1. Le navigateur envoie la demande à `POST /api/chat`.
2. Flask récupère la session et appelle `run_agent`.
3. `run_agent` vérifie d'abord s'il existe une action en attente de confirmation.
4. Le LLM reçoit le message, l'historique récent, le prompt système et la liste des tools.
5. Le LLM choisit `rechercher_concerts` avec `{"artiste": "Gims", "ville": "Décines"}`.
6. Python valide les arguments, exécute le tool et interroge Ticketmaster.
7. Le résultat du tool est renvoyé au LLM.
8. Le LLM rédige la réponse finale à partir de ce résultat.
9. La demande, la réponse et la trace du tool sont enregistrées dans SQLite.

Ce que voit l'utilisateur / ce qui se passe

Ce que voit l'utilisateur	Ce qui se passe réellement
Il tape une phrase en français.	Flask crée ou réutilise une session.
L'agent semble « réfléchir ».	Le LLM choisit un tool et ses arguments.
Des concerts apparaissent.	Python a exécuté Ticketmaster et renvoyé les données.
Une demande « Confirmer ? » s'affiche.	L'écriture est stockée dans `pending_actions`.
Il clique sur Oui.	Le tool d'écriture est exécuté et SQLite est modifié.
Il ouvre « Traces de l'agent ».	Les appels réels, leurs arguments et leurs durées sont affichés.

Les garde-fous

- **Pas d'invention** : le LLM reçoit les résultats des tools, pas une base de concerts imaginaire.
- **Validation** : les arguments sont vérifiés avant l'exécution.
- **Confirmation** : une écriture n'est jamais exécutée directement.
- **Traçabilité** : chaque appel laisse une trace.
- **Secours** : si le LLM est absent, `agent/regles.py` prend le relais.

---

5. Les 6 tools

Tool	Type	Exemple de demande	Ce qu'il fait
`consulter_preferences`	lecture	« Quels sont mes goûts ? »	Lit la ville, la région, le budget et les artistes suivis.
`rechercher_concerts`	lecture	« Gims à Décines »	Interroge Ticketmaster, filtre et formate les résultats.
`lister_favoris`	lecture	« Montre mes favoris »	Liste les concerts enregistrés.
`historique_agent`	lecture	« Montre l'historique »	Retourne les interactions et les traces récentes.
`ajouter_favori`	écriture	« Ajoute ce concert »	Enregistre un concert après confirmation.
`ajouter_artiste_suivi`	écriture	« Suis Ninho »	Ajoute un artiste à la liste suivie après confirmation.

Chaque tool possède une **description en français** dans `agent/tools.py`. C'est cette description, avec le schéma JSON des arguments, qui est envoyée au LLM. Le LLM voit donc le mode d'emploi du tool, pas son code Python.

Validation des écritures

`ajouter_favori` n'accepte pas un concert arbitraire : il vérifie que le `concert_id` provient du cache de la dernière recherche de la session. Cela empêche le LLM d'inventer un identifiant.

`ajouter_artiste_suivi` vérifie que le nom n'est pas vide. Les deux tools sont marqués `is_write=True` : le système sait qu'ils nécessitent une confirmation.

Pourquoi seulement 6 tools ?

Un agent fiable ne doit pas avoir accès à tout. Les tools sont limités aux actions utiles du projet : lire les préférences, chercher, lister, historiser et écrire deux types de données. C'est le principe du **moindre privilège**.

Recherche large et périodes

Quand aucun artiste n'est précisé, `rechercher_concerts` interroge **tous les artistes suivis**, et non plus seulement les huit premiers. Pour rester rapide et éviter de saturer Ticketmaster :

- la liste est plafonnée à **40 artistes** ; au-delà, l'agent indique que la recherche est partielle ;
- chaque artiste est cherché par mot-clé, avec une pause de 0,25 s entre deux appels ;
- la recherche s'arrête après **30 secondes** si Ticketmaster est trop lent, avec un avertissement ;
- le résultat est plafonné à **30 concerts** ; un avertissement indique combien de concerts existent avant la limite ;
- les erreurs de certains artistes sont signalées sans faire croire à une absence totale de concerts.

Les périodes relatives reconnues sont : `demain`, `cette_semaine`, `ce_weekend`, `mois_prochain` et `cette_annee`. « Cette année » correspond au **1er janvier – 31 décembre de l'année en cours**. Ticketmaster ne renvoyant que des événements à venir, les concerts passés ne sont pas affichés.

Quand un artiste est nommé explicitement, la recherche reste ciblée : l'application cherche d'abord l'identifiant exact de l'artiste, ce qui évite les homonymes et les résultats hors sujet.

---

6. Confirmation avant écriture

Séquence visible

```
Utilisateur : Ajoute le concert de Gims à Décines à mes favoris.
Agent       : Je vais ajouter « Gims – 02/11/2026 à Décines (LDLC Arena) » à tes favoris. Confirmer ?
Utilisateur : Oui
Agent       : C'est fait : « Gims – 02/11/2026 à Décines » a été ajouté à tes favoris.
```

Séquence technique

1. Le LLM demande `ajouter_favori`.
2. Le code reconnaît que le tool est une écriture.
3. L'action et ses arguments sont stockés dans `pending_actions`, avec le `session_id`.
4. La boucle s'arrête et renvoie une demande de confirmation.
5. Le frontend affiche les boutons **Oui, confirmer** et **Non, annuler**.
6. « Oui » envoie `{"confirm": "oui"}` et exécute l'action.
7. « Non » supprime l'action et écrit une trace `refuse`.

Pourquoi ce n'est pas seulement une phrase dans le prompt

Le LLM pourrait oublier une consigne. Ici, la barrière est dans le code Python : même si le modèle demande l'écriture, `run_agent` refuse de l'exécuter tant qu'une confirmation n'est pas présente. C'est une garantie technique, pas une simple recommandation.

7. Base SQLite

Fichier local : `data/concerts.db` (ignoré par Git).

Table	Contenu	À quoi ça sert
`preferences`	Ville, région, budget	Personnaliser les recherches.
`artistes_suivis`	Artistes uniques	Savoir pour qui chercher.
`favoris`	Concerts enregistrés	Retrouver les concerts choisis.
`interactions`	Messages user/assistant	Reconstruire l'historique.
`tool_traces`	Tool, arguments, résultat, durée	Prouver ce que l'agent a fait.
`pending_actions`	Action en attente	Bloquer une écriture avant confirmation.
`resultats_cache`	Derniers concerts trouvés	Valider un favori par son identifiant.
`concerts_demo`	Jeu de démonstration	Fonctionner sans Ticketmaster.
`spotify_sessions`	Jetons Spotify	Éviter de redemander la connexion.

Le schéma est créé par `db.init_db()` au démarrage. Le seed ajoute 9 artistes, des préférences Lyon / Auvergne-Rhône-Alpes / 60 € et 12 concerts de démonstration.

Le cache des recherches

Chaque recherche remplace le cache de la session. Le cache sert uniquement à retrouver un concert de la dernière recherche quand l'utilisateur demande de l'ajouter aux favoris. Ce n'est **pas** une synchronisation périodique : Ticketmaster est interrogé à chaque nouvelle recherche.

8. Le LLM

`agent/llm.py` est le **seul** endroit qui appelle un modèle. Le client utilise une API compatible OpenAI :

```
POST {LLM_BASE_URL}/chat/completions
```

Configuration actuelle :

```
LLM_PROVIDER=gemini
LLM_MODEL=gemini-3.5-flash-lite
LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai
GEMINI_API_KEY=...
```

Providers supportés par le code : Gemini, OpenAI, OpenRouter, Groq et Ollama.

Le client :

- envoie `tools` et `tool_choice="auto"` ;
- normalise les `tool_calls` (arguments en chaîne JSON, identifiants manquants) ;
- convertit `function_call` en `tool_calls` pour les anciens modèles ;
- lève `LLMError` en cas d'erreur HTTP ou de réponse illisible.

Prompt système

Le prompt interdit explicitement d'inventer des concerts et impose l'utilisation des tools pour toute demande de concerts. Il précise aussi que les tools d'écriture sont confirmés par le système.

Secours sans LLM

Si `obtenir_llm()` renvoie `None` (pas de clé) ou si le LLM lève `LLMError`, `run_rules` prend le relais. Le mode est indiqué dans la réponse (`regles` ou `regles_fallback`) et dans le badge de l'interface. L'application reste donc utilisable sans connexion à un modèle.

---

9. Intégration Spotify

1. `GET /login` génère un `state` aléatoire, le stocke dans la session signée Flask et redirige vers Spotify.
2. `GET /callback` vérifie le `state`, échange le code contre un jeton et récupère le profil.
3. Les jetons sont enregistrés dans `spotify_sessions` (SQLite), jamais dans le cookie.
4. Les top artistes et les écoutes récentes sont fusionnés, dédupliqués et ajoutés à `artistes_suivis`.
5. `GET /logout` supprime la session Spotify côté serveur.

Scopes demandés : `user-read-private`, `user-top-read`, `user-read-recently-played`.

```
SPOTIFY_CLIENT_ID=...
SPOTIFY_CLIENT_SECRET=...
REDIRECT_URI=http://127.0.0.1:5000/callback
```

L'URI doit être strictement identique à celle du dashboard Spotify. Spotify refuse l'alias `localhost` : en local, utiliser `http://127.0.0.1:5000/callback`. En production, utiliser `https://<service>.onrender.com/callback`. En mode développement, Spotify limite l'accès à 5 comptes ajoutés dans **User Management**.

Le refresh token est conservé quand Spotify n'en renvoie pas lors d'un rafraîchissement. Si le refresh échoue, la session est supprimée et l'utilisateur doit se reconnecter. L'import est limité à 30 artistes pour ne pas saturer la barre latérale.

10. API HTTP

Méthode	Route	Rôle
`GET`	`/`	Interface.
`POST`	`/api/chat`	`{message}` ou `{confirm:"oui"/"non"}`.
`GET`	`/api/historique`	Interactions et traces de la session.
`GET`	`/api/preferences`	Préférences, artistes, favoris.
`POST`	`/api/preferences`	Met à jour ville, région, budget.
`GET`	`/api/artistes`	Liste des artistes suivis.
`POST`	`/api/artistes`	Ajoute un artiste.
`DELETE`	`/api/artistes/<nom>`	Retire un artiste.
`GET`	`/api/favoris`	Liste des favoris avec toutes leurs informations.
`DELETE`	`/api/favoris/<id>`	Retire un favori.
`GET`	`/api/etat`	État du LLM, de Ticketmaster, de Spotify et de l'action en attente.
`GET`	`/login`	Redirection OAuth Spotify.
`GET`	`/callback`	Retour OAuth, stockage des jetons et import.
`GET`	`/logout`	Déconnexion Spotify.
`GET`	`/api/spotify/status`	Configuration, connexion et profil Spotify.
`GET`	`/api/spotify/top-artists`	Top artistes Spotify de l'utilisateur.
`POST`	`/api/spotify/import`	Importe les artistes Spotify (`{limit:30}`).

Réponse type de `/api/chat` :

```
{
  "reply": "J'ai trouvé 2 concerts : ...",
  "traces": [
    {
      "tool": "rechercher_concerts",
      "arguments": {"artiste": "Gims", "ville": "Décines"},
      "statut": "succes",
      "duree_ms": 338,
      "resume": "2 concert(s), source=ticketmaster"
    }
  ],
  "confirmation": null,
  "mode": "llm"
}
```

11. Fiche détaillée d'un favori

Dans la colonne **Favoris**, chaque concert est cliquable. Une fiche affiche :

- l'artiste et la date ;
- l'heure ;
- la salle ;
- la ville, la région et le pays ;
- le prix minimum et maximum, ou « non communiqué » ;
- la source (`ticketmaster` ou `demo`) ;
- la date d'ajout ;
- le lien de billetterie lorsqu'il existe.

La fiche est alimentée par `GET /api/favoris` et **ne modifie pas la base**. Elle se ferme avec la croix, la touche `Échap` ou un clic à l'extérieur. Le bouton « retirer » reste séparé pour éviter les suppressions accidentelles.

12. Données de démonstration

Le jeu de démonstration contient notamment :

- Gims à Lyon (Halle Tony Garnier), le mois prochain ;
- Sofiane Pamart à Grenoble (Summum), le mois prochain ;
- Ninho à Lyon (Halle Tony Garnier), le mois prochain ;
- plusieurs concerts à Paris, Lille, Marseille et Toulouse à des dates décalées.

Les dates sont recalculées à chaque démarrage (`_next_month_day` , `_shift`) pour que la démo reste cohérente quelle que soit la date. Toute réponse issue de ce jeu contient `source="demo"` et un avertissement visible.

Ces données servent à tester l'interface, les tools et les scénarios sans dépendre du réseau. Elles ne représentent pas de vrais événements.

13. Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Les tests utilisent une base temporaire (`CONCERT_DB_PATH`) et forcent `MODE_DEMO=1` pour ne pas dépendre du réseau. Ils vérifient :

- la présence des 6 tools et des 2 tools d'écriture ;
- le mode règles sans clé LLM ;

- le scénario 1 (3 concerts en Auvergne-Rhône-Alpes) ;
- le scénario 2 (aucun résultat, aucune invention) ;
- le scénario 3 (confirmation puis ajout en base) ;
- le refus de confirmation ;
- l'historique et les traces ;
- l'effacement du budget ;
- les routes artistes et favoris ;
- la période « cette année » et la recherche large sur tous les artistes ;
- Spotify non configuré, redirection OAuth, état invalide et import refusé ;
- import Spotify réussi, déduplication et ajout dans les artistes suivis ;
- stockage, conservation du refresh token et suppression de session Spotify.

Résultat attendu : `Ran 15 tests ... OK`.

Les tests ne remplacent pas une vérification manuelle des appels réels Ticketmaster et Spotify. Ils garantissent le comportement logique, la validation et les scénarios.

---

14. Sécurité et limites

- Les secrets sont dans `.env`, ignoré par Git. Ne jamais committer `.env`.
- `FLASK_SECRET` doit être changé pour autre chose qu'une valeur de démonstration.
- Les concerts de démonstration ne sont pas réels.
- La couverture Ticketmaster en France est partielle. La recherche utilise `locale=fr-fr` puis l'identifiant exact de l'attraction pour éviter les homonymes ; si aucun événement n'est trouvé, les données de démonstration sont affichées et signalées.
- Spotify : les jetons sont en base, pas dans le cookie ; le `state` OAuth est vérifié ; les scopes restent limités au profil, aux tops et aux écoutes récentes.
- Le mode développement Spotify est limité à 5 comptes ; les autres recevront une erreur 403.
- L'application écoute sur `127.0.0.1` en local ; `HOST=0.0.0.0` est prévu pour Render/Gunicorn.
- Le LLM réel a été validé avec `gemini-3.5-flash-lite` ; il nécessite une clé Gemini et un redémarrage du serveur.
- La base SQLite Render gratuite est éphémère ; utiliser un disque persistant pour conserver les données.
- Aucune notification automatique, aucun paiement et aucun achat de billet ne sont implémentés. Les liens ouvrent Ticketmaster.

---

15. Déploiement

Le projet contient `render.yaml` et `Procfile`. La procédure détaillée se trouve dans le `README.md`.

Points essentiels :

1. Le dépôt GitHub doit contenir `app.py`, `render.yaml`, `requirements.txt`, `agent/`, `static/` et `templates/` à sa racine.
2. Render lance `pip install -r requirements.txt`, puis Gunicorn avec `app:app`.
3. Les variables `GEMINI_API_KEY`, `TICKETMASTER_API_KEY`, `SPOTIFY_CLIENT_ID`, `SPOTIFY_CLIENT_SECRET` et `REDIRECT_URI` doivent être renseignées dans Render.
4. `REDIRECT_URI` doit être identique dans Render et dans Spotify Dashboard.
5. Le plan gratuit remet la base SQLite à zéro à chaque redémarrage.

---

16. Checklist de conformité

Consigne du projet	Où le vérifier	État
Application en Python	`app.py`, `requirements.txt`, `render.yaml`	Conforme
Interface Flask	`templates/index.html`, `static/`	Conforme
LLM local ou distant	`agent/llm.py`, badge LLM, `/api/etat`	Conforme
Le LLM choisit les tools	`agent/loop.py`, panneau « Traces »	Conforme
SQLite lue et écrite	`db.py`, tools d'écriture, tests	Conforme
Confirmation avant modification	`pending_actions`, boutons Oui/Non, tests	Conforme
Historique des interactions	table `interactions`, `/api/historique`	Conforme
Traces des tools	table `tool_traces`, panneau « Traces »	Conforme
3 scénarios	`DEMOS.md` / `DEMOS.pdf`, tests	Conforme
4 livrables	README, documentation, présentation, démos + PDFs	Conforme

Le retour qui évoque Node.js, Express, `localStorage` et l'absence de LLM concerne l'ancien projet `concert-alert`. Le projet livré ici est `concert-agent-code`.

---

17. FAQ technique

Est-ce vraiment un agent ?

Oui. Le LLM ne se contente pas de répondre : il choisit un tool, Python l'exécute, le résultat revient au LLM, puis le LLM poursuit ou répond. La trace montre le tool et ses arguments.

Que se passe-t-il sans clé Gemini ?

Le mode règles prend le relais. Les tools, SQLite, les favoris et les traces continuent de fonctionner.

Que se passe-t-il si Ticketmaster tombe ?

L'erreur est signalée. Si `DEMO_FALLBACK=1`, l'application utilise les données de démonstration et l'indique. Si le fallback est désactivé, l'agent renvoie une erreur explicite.

Pourquoi les prix sont-ils souvent absents ?

Ticketmaster ne fournit pas toujours `priceRanges`. L'application affiche alors `Prix : non communiqué` au lieu d'inventer un prix.

Pourquoi certains artistes n'ont aucun concert ?

La couverture de Ticketmaster en France est partielle. L'absence de résultat ne signifie pas que l'artiste ne fait jamais de concert.

Les données se mettent-elles à jour automatiquement ?

Non. Chaque recherche interroge Ticketmaster en direct. Il n'y a pas de tâche de fond toutes les cinq minutes. Un favori est un instantané.

Puis-je utiliser un autre LLM ?

Oui. `agent/llm.py` accepte Gemini, OpenAI, OpenRouter, Groq et Ollama via `LLM_PROVIDER`, `LLM_BASE_URL` et `LLM_MODEL`.

Pourquoi SQLite plutôt qu'une grosse base ?

SQLite suffit pour un projet mono-utilisateur, ne demande aucun serveur et garde les données dans un fichier. Pour plusieurs utilisateurs simultanés, Postgres serait plus adapté.

Où voir ce que l'agent a fait ?

Dans le panneau « Traces de l'agent » sous chaque réponse, ou via `GET /api/historique`.